

Міністерство освіти і науки України
Національний технічний університет
України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет прикладної математики

Кафедра системного програмування і спеціалізованих комп'ютерних систем

Лабораторна робота № 3

з дисципліни

«Моделювання та оптимізація комп'ютерних систем»

Тема: «МОДЕЛЮВАННЯ КЛІТИННИХ АВТОМАТІВ»

Виконала студентка групи:

КВ-11 Михайліченко Софія

Перевірив: Малежик П.М.

Оцінка:

Київ – 2024

Загальне завдання

1. Опрацювати теоретичну частину.
2. Ознайомитись із роботою програми *Life* в *MatLab*.
3. Вивчити роботу програми *Conway's life*.
4. Для «хаотичної» конфігурації, в якій кожна клітина знаходиться в стані 1 з імовірністю 50% розглянути часову еволюцію правил 00010010 (правила 18), правила 01001001 (правила 73) и правила 10001000 (правила 136).
5. Для "хаотичної" початкової конфігурації кожній клітині розглянути часову еволюцію правила Вольфрама з номером, який дорівнює номеру у списку групи помноженому на 5 ($14 * 5 = 70$).

Завдання 1

Перед виконанням будь-якого завдання, детально розглянемо теоретичні відомості та випишемо найголовніше з них.

Моделювання клітинних автоматів є однією з ключових галузей досліджень у області комп'ютерних наук, відомій як дискретні динамічні системи. Клітинні автомати становлять прості, але потужні моделі, які дозволяють розглядати поведінку складних систем через їхню взаємодію на рівні окремих компонентів, або "клітин".

Основні аспекти моделювання клітинних автоматів включають:

1. **Класифікація правил:** Правила клітинних автоматів визначають, як стани клітин змінюються від одного кроку до наступного. Ці правила можуть бути класифіковані за різними критеріями, такими як кількість сусідів, які враховуються, або типи перетворень, які вони виконують.
2. **Властивості і емерджентність:** Клітинні автомати часто використовуються для дослідження емерджентних властивостей систем, що виникають в результаті взаємодії багатьох простих компонентів. Це дозволяє вивчати явища самоорганізації і формування складних структур з простих правил взаємодії.
3. **Прикладні застосування:** Клітинні автомати використовуються в різних прикладних задачах, таких як моделювання процесів в біології (наприклад, росту рослин або розвитку тканин), фізиці (наприклад, моделювання фазових переходів) та соціальних наук (наприклад, моделювання поширення хвороб або формування громадської думки).
4. **Алгоритми та обчислення:** Клітинні автомати можуть бути використані для розв'язання обчислювальних задач, таких як симуляція або

оптимізація. Вони є потужним інструментом для моделювання складних систем і аналізу їхніх властивостей.

5. **Графічне візуалізація і аналіз:** Для розуміння та візуалізації поведінки клітинних автоматів часто використовуються різні графічні методи. Це дозволяє дослідникам спостерігати за динамікою систем та аналізувати їхні властивості.

Моделювання клітинних автоматів є потужним інструментом для вивчення різноманітних системних явищ і використовується в різних галузях науки та інженерії.

Завдання 2

Розглянемо роботу програми Life в MatLab. Для цього нам потрібно детальніше ознайомитись з правилами даної гри та після детального аналізу приступити до ознайомленням написаної гри в MatLab.

Гра Конвея "Життя" - типовий приклад клітинного автомата, як математичного об'єкта, що представляє собою дискретну динамічну систему. За суттю клітинні автомати є синтетичними світами, поведінка яких здебільшого визначається простими локально діючими правилами.

У цих світах простір являє собою рівномірну сітку, кожна клітинка якої (клітка) містить інформацію про свій стан. Час є дискретним. Закони такого світу являють собою невелику кількість правил, основні з яких описуються таблицею переходів, за якою клітина обчислює свій новий стан на кожному такті (мінімальний відрізок часу) на основі свого стану і станів її сусідів.

Індивідуум цієї популяції представлений клітиною в стані 1, у той час як клітина в стані 0 представляє порожній простір (для образності можна говорити про "живі" і "мертві" клітини). Мірою течії часу служить зміна поколінь колонії, яка відбувається за відомими правилами.

Популяція (або колонія) клітин в "Житті" може весь час зростати, безперервно міняючи своє розташування, форму і число клітин. Однак частіше колонія стає врешті-решт стаціонарною або циклічно повторює один і той же кінцевий набір станів. Довжина циклу називається періодом колонії.

Правила гри Конвея "Життя"

Дія гри відбувається на певній площині, розділеній на клітки. Кожна клітина оточена 8 такими ж клітинами (околиця Мура). Кожна клітина може знаходитися в двох станах - живому чи мертвому, тобто порожньому. На стан будь-якої клітини впливають стани сусідніх клітин. В часі ці стани є дискретними відповідно до певних правил або генетичних законів Конвея, що складаються з 2 пунктів:

- 1) Виживання або загибель. Якщо жива клітина має менше 2 або більше 3 сусідів в околиці з 8 клітин, то в наступному поколінні вона вмирає (моделювання реальних умов - нестача харчування або перенаселеність), в іншому випадку вона виживає;
- 2) Народження. У порожній клітці з'являється жива клітина, якщо у вихідної клітини рівно 3 сусіда.

Загибель і народження всіх організмів відбувається одночасно.

Ситуації, що виникають в грі схожі на реальні процеси, що відбуваються при зародженні, розвитку і загибелі живих організмів

Гра "Життя" відноситься до категорії так званих моделюючих ігор - ігор, які імітують процеси, що відбуваються в реальному житті.

Основна ідея гри полягає в тому, щоб, почавши з якого-небудь простого розташування живих клітин, простежити за еволюцією вихідної позиції під дією згадуваних генетичних законів Конвея, які управляють народженням, загибеллю і виживанням клітин.

Генетичні закони Конвея задовольняють трьом основним умовам:

- 1) не повинно бути жодної вихідної конфігурації, для якої існувало б просте доведення можливості необмеженого росту популяції;
- 2) повинні існувати такі початкові конфігурації, які свідомо мають здатність безмежно розвиватися;
- 3) повинні існувати прості початкові конфігурації, які протягом значного проміжку часу зростають, зазнають різноманітні зміни і закінчують свою еволюцію в один з наступних трьох способів:

- а. повністю зникають;
- б. переходять у стійку конфігурацію і перестають змінюватися взагалі;
- с. виходять на коливальний режим з певним періодом.

Популяція клітин невинно зазнає незвичайні зміни.

Завдання 3

Код програми Conway's life заданий у методичних вказівках:

```
%Conway's life with GUI

clf

clear all

%=====

%build the GUI

%define the plot button

plotbutton=uicontrol('style','pushbutton',...

    'string','Run', ...

    'fontsize',12, ...

    'position',[100,400,50,20], ...

    'callback','run=1;');

%define the stop button

erasebutton=uicontrol('style','pushbutton',...

    'string','Stop', ...

    'fontsize',12, ...

    'position',[200,400,50,20], ...

    'callback','freeze=1;');

%define the Quit button
```

```

quitbutton=icontrol('style','pushbutton',...

    'string','Quit', ...

    'fontsize',12, ...

    'position',[300,400,50,20], ...

    'callback','stop=1;close;');

```

```

number = uicontrol('style','text', ...

    'string','1', ...

    'fontsize',12, ...

    'position',[20,400,50,20]);

```

```

%=====

```

```

%CA setup

```

```

n=128;

```

```

%initialize the arrays

```

```

z = zeros(n,n);

```

```

cells = z;

```

```

sum = z;

```

```

%set a few cells to one

```

```

cells(n/2,.25*n:.75*n) = 1;

```

```

cells(.25*n:.75*n,n/2) = 1;

```

```

%cells(.5*n-1,.5*n-1)=1;

```

```

%cells(.5*n-2,.5*n-2)=1;

```

```

%cells(.5*n-3,.5*n-3)=1;

```

```

cells = (rand(n,n))<.5 ;

```

```
%how long for each case to stability or simple oscillators
```

```
%build an image and display it
```

```
imh = image(cat(3,cells,z,z));
```

```
set(imh, 'erasemode', 'none')
```

```
axis equal
```

```
axis tight
```

```
%index definition for cell update
```

```
x = 2:n-1;
```

```
y = 2:n-1;
```

```
%Main event loop
```

```
stop= 0; %wait for a quit button push
```

```
run = 0; %wait for a draw
```

```
freeze = 0; %wait for a freeze
```

```
while (stop==0)
```

```
    if (run==1)
```

```
        %nearest neighbor sum
```

```
        sum(x,y) = cells(x,y-1) + cells(x,y+1) + ...
```

```
            cells(x-1, y) + cells(x+1,y) + ...
```

```
            cells(x-1,y-1) + cells(x-1,y+1) + ...
```

```
            cells(3:n,y-1) + cells(x+1,y+1);
```

```
        % The CA rule
```

```
        cells = (sum==3) | (sum==2 & cells);
```

```
        %draw the new image
```

```
        set(imh, 'cdata', cat(3,cells,z,z) )
```

```

        %update the step number display

        stepnumber = 1 + str2num(get(number,'string'));

        set(number,'string',num2str(stepnumber))

    end

    if (freeze==1)

        run = 0;

        freeze = 0;

    end

    drawnow %need this in the loop for controls to work

end

```

Для виконання даного завдання спочатку розглянемо загальний принцип роботи програми, а потім перейдемо до аналізу кожної частини реалізації гри:

Програма реалізує класичну гру "Життя" (Conway's Game of Life) створену Джоном Конвеєм, яка представляє собою клітинний автомат, що працює на простому правилі, що застосовується до кожної клітини сітки. Головний принцип полягає в еволюції стану сітки залежно від стану сусідніх клітин на попередньому кроці, що відбувається в безкінечному циклі, поки користувач не натисне кнопку "Quit". Початковий стан сітки генерується випадковим чином або встановлюється вручну. Користувач має можливість запустити еволюцію сітки натисканням кнопки "Run", зупинити її натисканням кнопки "Stop" або закрити програму натисканням кнопки "Quit". Кожна ітерація циклу оновлює стан кліток відповідно до правил "Життя", заснованих на кількості живих сусідів кожної клітки. Зображення сітки оновлюється на кожній ітерації для відображення нового стану. Програма використовує GUI з трьома кнопками: "Run", "Stop" і "Quit", а також відображенням номера поточного кроку еволюції. Код програми реалізований в середовищі MATLAB з використанням функцій **uicontrol** для створення GUI і графічних функцій для відображення сітки. Цей алгоритм гарантує безкінечну еволюцію сітки згідно з правилами "Життя" до тих

пiр, поки користувач не вирiшить завершити програму. Такий пiдхiд дозволяє вiдтворити рiзноманiтнi структури i змiни в грі, вiд простих форм до складних життєвих циклiв.

Детальний аналіз викорисних функцій в MatLab для реалізації гри:

clf: Очищає попередні графіки з фігур, щоб створити новий графічний вікно.

clear all: Очищає всі змінні з робочого простору, щоб забезпечити чисте середовище виконання.

Коментарі *%build the GUI, %define the plot button, %define the stop button, %define the Quit button*: Пояснюють, що наступні рядки коду створюють елементи керування для GUI - кнопки "Run", "Stop" та "Quit".

Створення кнопки "Run":

`plotbutton=uicontrol('style','pushbutton',...:` Створює кнопку з властивостями `pushbutton`.

`'string','Run', ...:` Встановлює текст кнопки на "Run".

`'callback', 'run=1;':` Встановлює функцію зворотного виклику для кнопки, яка встановлює змінну `run` в 1.

Створення кнопки "Stop": Аналогічно до створення кнопки "Run", але з функцією зворотного виклику, яка встановлює `freeze` в 1.

Створення кнопки "Quit": Аналогічно до створення інших кнопок, але з функцією зворотного виклику, яка закриває вікно гри.

Створення елемента тексту "number": Відображає номер кроку гри.

Коментар *%CA setup*: Позначає початок налаштувань клітинного автомата (CA). `n=128;:` Встановлює розмір сітки 128x128.

Ініціалізація масивів:

`z = zeros(n,n);:` Створює матрицю нулів для подальшого використання.

`cells = z;:` Ініціалізує початковий стан клітин.

`sum = z;:` Ініціалізує матрицю суми для обчислення наступного стану клітини.

Встановлення початкових живих клітин:

`cells(n/2,25*n:.75*n) = 1;:` Встановлює горизонтальну смугу живих клітин.

`cells(.25*n:.75*n,n/2) = 1;` Встановлює вертикальну смугу живих клітин.
`cells = (rand(n,n))<.5;` Встановлює випадковий початковий стан сітки, де клітини випадково обираються як живі або мертві за випадковими значеннями.

Створення графічного вікна та відображення сітки:

`imh = image(cat(3,cells,z,z));` Створює графічний об'єкт для відображення сітки.

`set(imh, 'erasemode', 'none');` Встановлює режим оновлення графіки, щоб уникнути очищення графічного вікна при оновленні.

`axis equal, axis tight;` Встановлює пропорційність осей та межі графіку.

Визначення індексів для оновлення клітин:

`x = 2:n-1; y = 2:n-1;` Визначає внутрішню частину сітки для оновлення.

Основний цикл подій:

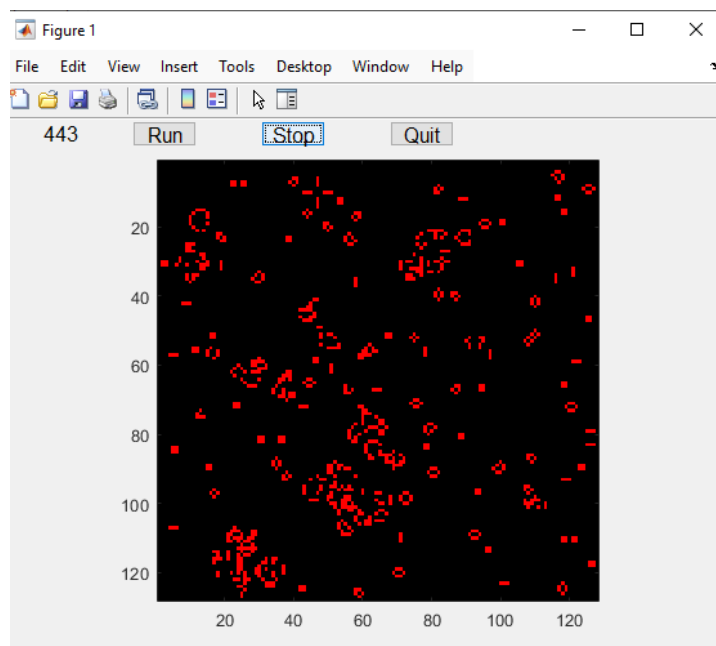
Виконується безкінечно, поки змінна `stop` не буде встановлена в 1.

Перевіряється, чи натиснута кнопка "Run" (`run==1`), і оновлюється стан клітин.

Перевіряється, чи натиснута кнопка "Stop" (`freeze==1`), і встановлюється змінна `run` в 0.

Функція `drawnow` викликається для оновлення графіки.

Приклад роботи даної програми:



Завдання 4

Виконаємо детальний аналіз кожного правила перед написанням програми.

Правило 18(00010010)

Спочатку зрозуміємо, як працює правило 18 для еволюції клітинного автомата. Правило 18 стверджує, що значення кожної центральної клітини на кроці $t + 1$ залежить від її власного значення і значень двох сусідніх клітин на кроці t .

Воно представлене у двійковій системі як 00010010. Кожна позиція в цьому числі відповідає конкретній можливій комбінації трьох сусідніх клітин на кроці t , а відповідне значення (0 або 1) показує, яке значення матиме центральна клітина на кроці $t + 1$.

Таким чином, для правила 18 таблиця буде виглядати наступним чином:

Можливі комбінації трьох сусідніх клітин на кроці t	Значення центральної клітини на кроці $t + 1$
111	0
110	0
101	0
100	1
011	0
010	0
001	1
000	0



Код програми для правила 18:

```
function new_state = apply_custom_rule(neighbors)
    % Custom Rule (Default: Rule 18)
    if neighbors(1) == 1 && neighbors(2) == 1 && neighbors(3) == 1
        new_state = 0;
    elseif neighbors(1) == 1 && neighbors(2) == 1 && neighbors(3) == 0
```

```

        new_state = 0;
    elseif neighbors(1) == 1 && neighbors(2) == 0 && neighbors(3) == 1
        new_state = 0;
    elseif neighbors(1) == 1 && neighbors(2) == 0 && neighbors(3) == 0
        new_state = 1;
    elseif neighbors(1) == 0 && neighbors(2) == 1 && neighbors(3) == 1
        new_state = 0;
    elseif neighbors(1) == 0 && neighbors(2) == 1 && neighbors(3) == 0
        new_state = 0;
    elseif neighbors(1) == 0 && neighbors(2) == 0 && neighbors(3) == 1
        new_state = 1;
    else
        new_state = 0;
    end
end

% GUI Initialization
clf
clear all

% Build the GUI
plotbutton = uicontrol('style','pushbutton',...
    'string','Run', ...
    'fontsize',12, ...
    'position',[100,400,50,20], ...
    'callback','run=1;');

erasebutton = uicontrol('style','pushbutton',...
    'string','Stop', ...
    'fontsize',12, ...
    'position',[200,400,50,20], ...
    'callback','freeze=1;');

quitbutton = uicontrol('style','pushbutton',...
    'string','Quit', ...
    'fontsize',12, ...
    'position',[300,400,50,20], ...
    'callback','stop=1;close;');
number = uicontrol('style','text', ...
    'string','1', ...
    'fontsize',12, ...
    'position',[20,400,50,20]);

% CA setup
n = 50; % Збільшення розмірів клітин
z = zeros(n,n);
cells = z;
sum = z;
cells(n/2, .25*n:.75*n) = 1;
cells(.25*n:.75*n, n/2) = 1;
cells = (rand(n,n)) < 0.5;

imh = image(cat(3, cells, cells, z)); % Сині клітини
set(imh, 'erasemode', 'none')
axis equal
axis tight

x = 2:n-1;
y = 2:n-1;

stop = 0;
run = 0;

```

```

freeze = 0;
stepnumber = 1;

while (stop == 0 && stepnumber <= 50)

    if (run == 1)
        updated_config = zeros(size(cells));
        for i = 1:n
            for j = 1:n
                neighbors = [cells(mod(i-2, n)+1, j), ...
                             cells(i, mod(j, n)+1), ...
                             cells(mod(i, n)+1, mod(j, n)+1)];
                updated_config(i, j) = apply_custom_rule(neighbors);
            end
        end
        cells = updated_config;

        set(imh, 'cdata', cat(3, cells, cells, z));

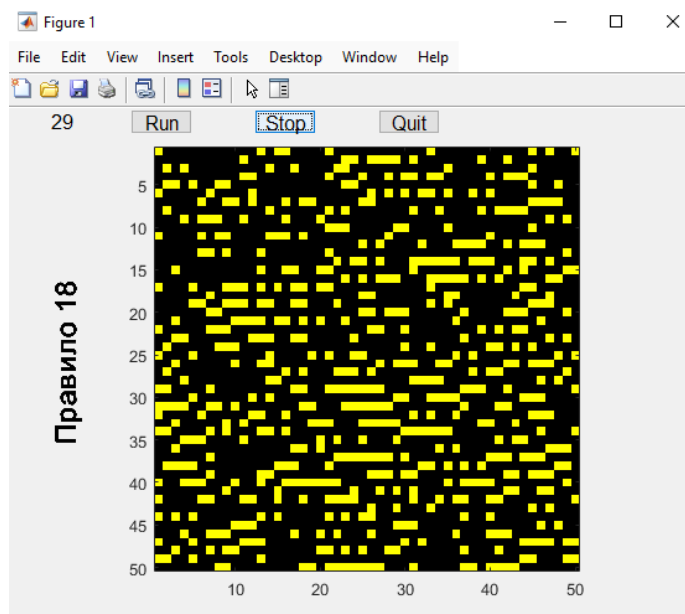
text(-10, 35.5, 'П', 'Rotation', 90, 'Color', 'k', 'FontSize', 17);
text(-10, 33, 'р', 'Rotation', 90, 'Color', 'k', 'FontSize', 17);
text(-10, 31, 'а', 'Rotation', 90, 'Color', 'k', 'FontSize', 17);
text(-10, 29, 'в', 'Rotation', 90, 'Color', 'k', 'FontSize', 17);
text(-10, 27, 'и', 'Rotation', 90, 'Color', 'k', 'FontSize', 17);
text(-10, 25, 'л', 'Rotation', 90, 'Color', 'k', 'FontSize', 17);
text(-10, 23, 'о', 'Rotation', 90, 'Color', 'k', 'FontSize', 17);
text(-10, 20, '18', 'Rotation', 90, 'Color', 'k', 'FontSize', 17);

        stepnumber = 1 + str2num(get(number, 'string'));
        set(number, 'string', num2str(stepnumber))
        pause(0.1);
    end

    if (freeze == 1)
        run = 0;
        freeze = 0;
    end
    drawnow
end

```

Виконання програми для правила 18:



Правило 73(01001001)

Можливі комбінації трьох сусідніх клітин на кроці t	Значення центральної клітини на кроці t + 1
111	0
110	1
101	0
100	0
011	1
010	0
001	0
000	1

Rule icon



■ 1 | □ 0

Код програми для правила 73:

```
function new_state = apply_custom_rule(neighbors)
    % Custom Rule (Default: Rule 73)
    if neighbors(1) == 1 && neighbors(2) == 1 && neighbors(3) == 1
        new_state = 0;
    elseif neighbors(1) == 1 && neighbors(2) == 1 && neighbors(3) == 0
        new_state = 1;
    elseif neighbors(1) == 1 && neighbors(2) == 0 && neighbors(3) == 1
        new_state = 0;
    elseif neighbors(1) == 1 && neighbors(2) == 0 && neighbors(3) == 0
        new_state = 0;
    elseif neighbors(1) == 0 && neighbors(2) == 1 && neighbors(3) == 1
        new_state = 1;
    elseif neighbors(1) == 0 && neighbors(2) == 1 && neighbors(3) == 0
        new_state = 0;
    elseif neighbors(1) == 0 && neighbors(2) == 0 && neighbors(3) == 1
        new_state = 0;
    else
        new_state = 1;
    end
end

% GUI Initialization
clf
clear all

% Build the GUI
```

```

plotbutton = uicontrol('style','pushbutton',...
    'string','Run', ...
    'fontsize',12, ...
    'position',[100,400,50,20], ...
    'callback','run=1;');

erasebutton = uicontrol('style','pushbutton',...
    'string','Stop', ...
    'fontsize',12, ...
    'position',[200,400,50,20], ...
    'callback','freeze=1;');

quitbutton = uicontrol('style','pushbutton',...
    'string','Quit', ...
    'fontsize',12, ...
    'position',[300,400,50,20], ...
    'callback','stop=1;close;');
number = uicontrol('style','text', ...
    'string','1', ...
    'fontsize',12, ...
    'position',[20,400,50,20]);

% CA setup
n = 50; % Збільшення розмірів клітин
z = zeros(n,n);
cells = z;
sum = z;
cells(n/2, .25*n:.75*n) = 1;
cells(.25*n:.75*n, n/2) = 1;
cells = (rand(n,n)) < 0.5;

imh = image(cat(3, cells, cells, z)); % Сині клітини
set(imh, 'erasemode', 'none')
axis equal
axis tight

x = 2:n-1;
y = 2:n-1;

stop = 0;
run = 0;
freeze = 0;
stepnumber = 1;

while (stop == 0 && stepnumber <= 50)

    if (run == 1)
        updated_config = zeros(size(cells));
        for i = 1:n
            for j = 1:n
                neighbors = [cells(mod(i-2, n)+1, j), ...
                    cells(i, mod(j, n)+1), ...
                    cells(mod(i, n)+1, mod(j, n)+1)];
                updated_config(i, j) = apply_custom_rule(neighbors);
            end
        end
        cells = updated_config;

        set(imh, 'cdata', cat(3, cells, cells, z));

    text(-10, 35.5, 'П', 'Rotation', 90, 'Color', 'k', 'FontSize', 17);
    text(-10, 33, 'p', 'Rotation', 90, 'Color', 'k', 'FontSize', 17);

```

```

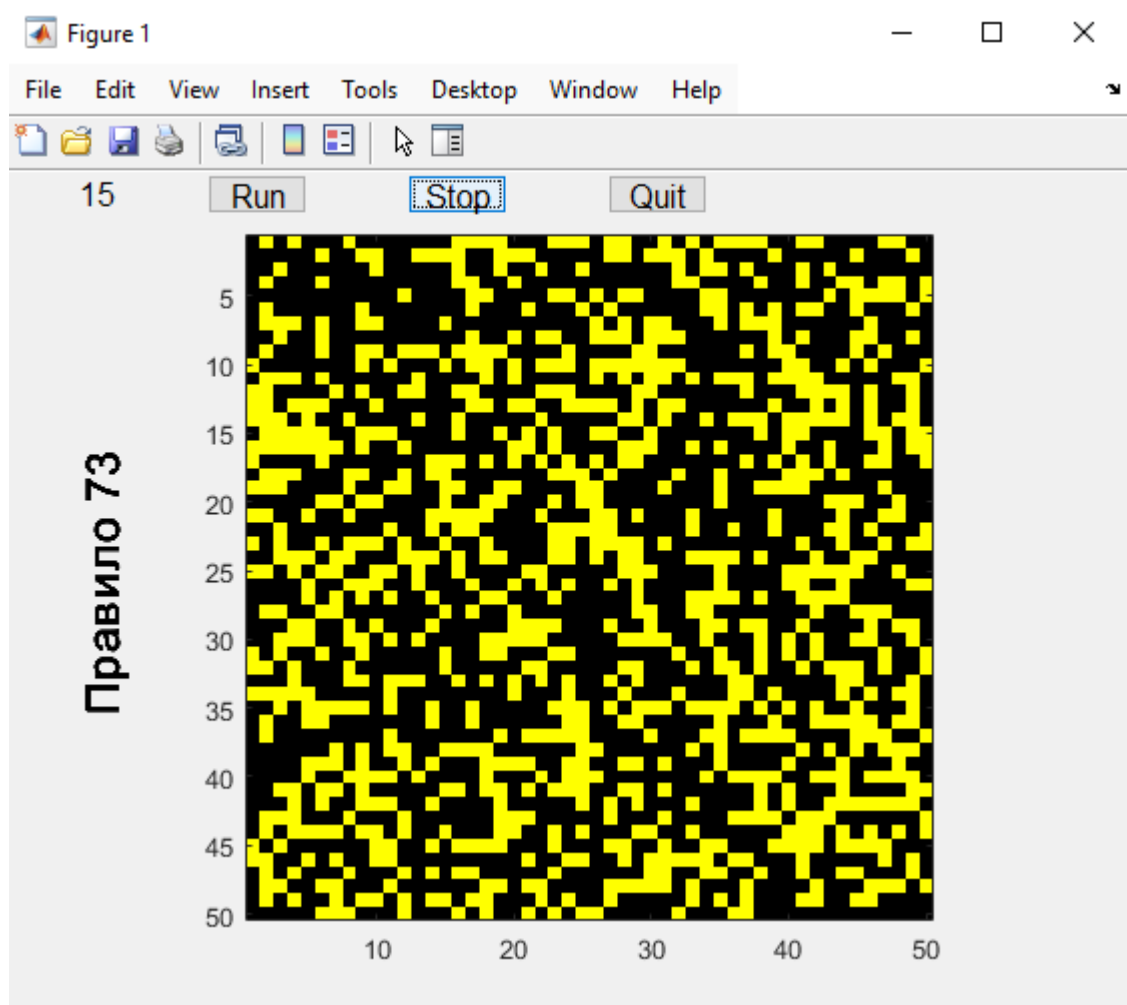
text(-10, 31, 'a', 'Rotation', 90, 'Color', 'k', 'FontSize', 17);
text(-10, 29, 'в', 'Rotation', 90, 'Color', 'k', 'FontSize', 17);
text(-10, 27, 'и', 'Rotation', 90, 'Color', 'k', 'FontSize', 17);
text(-10, 25, 'л', 'Rotation', 90, 'Color', 'k', 'FontSize', 17);
text(-10, 23, 'о', 'Rotation', 90, 'Color', 'k', 'FontSize', 17);
text(-10, 20, '73', 'Rotation', 90, 'Color', 'k', 'FontSize', 17);

    stepnumber = 1 + str2num(get(number, 'string'));
    set(number, 'string', num2str(stepnumber))
    pause(0.1);
end

if (freeze == 1)
    run = 0;
    freeze = 0;
end
drawnow
end

```

Виконання програми для правила 73:



Правило 136(10001000)

Можливі комбінації трьох сусідніх клітин на кроці t	Значення центральної клітини на кроці t + 1
111	1
110	0
101	0
100	0
011	1
010	0
001	0
000	0

Rule icon



■ 1 | □ 0

Код програми для правила 136:

```
function new_state = apply_custom_rule(neighbors)
    % Custom Rule (Default: Rule 136)
    if neighbors(1) == 1 && neighbors(2) == 1 && neighbors(3) == 1
        new_state = 1;
    elseif neighbors(1) == 1 && neighbors(2) == 1 && neighbors(3) == 0
        new_state = 0;
    elseif neighbors(1) == 1 && neighbors(2) == 0 && neighbors(3) == 1
        new_state = 0;
    elseif neighbors(1) == 1 && neighbors(2) == 0 && neighbors(3) == 0
        new_state = 0;
    elseif neighbors(1) == 0 && neighbors(2) == 1 && neighbors(3) == 1
        new_state = 1;
    elseif neighbors(1) == 0 && neighbors(2) == 1 && neighbors(3) == 0
        new_state = 0;
    elseif neighbors(1) == 0 && neighbors(2) == 0 && neighbors(3) == 1
        new_state = 0;
    else
        new_state = 0;
    end
end

% GUI Initialization
clf
clear all

% Build the GUI
plotbutton = uicontrol('style','pushbutton',...
```

```

    'string','Run', ...
    'fontsize',12, ...
    'position',[100,400,50,20], ...
    'callback','run=1;');

erasebutton = uicontrol('style','pushbutton',...
    'string','Stop', ...
    'fontsize',12, ...
    'position',[200,400,50,20], ...
    'callback','freeze=1;');

quitbutton = uicontrol('style','pushbutton',...
    'string','Quit', ...
    'fontsize',12, ...
    'position',[300,400,50,20], ...
    'callback','stop=1;close;');
number = uicontrol('style','text', ...
    'string','1', ...
    'fontsize',12, ...
    'position',[20,400,50,20]);

% CA setup
n = 50; % Збільшення розмірів клітин
z = zeros(n,n);
cells = z;
sum = z;
cells(n/2, .25*n:.75*n) = 1;
cells(.25*n:.75*n, n/2) = 1;
cells = (rand(n,n)) < 0.5;

imh = image(cat(3, cells, cells, z)); % Сині клітини
set(imh, 'erasemode', 'none')
axis equal
axis tight

x = 2:n-1;
y = 2:n-1;

stop = 0;
run = 0;
freeze = 0;
stepnumber = 1;

while (stop == 0 && stepnumber <= 50)

    if (run == 1)
        updated_config = zeros(size(cells));
        for i = 1:n
            for j = 1:n
                neighbors = [cells(mod(i-2, n)+1, j), ...
                    cells(i, mod(j, n)+1), ...
                    cells(mod(i, n)+1, mod(j, n)+1)];
                updated_config(i, j) = apply_custom_rule(neighbors);
            end
        end
        cells = updated_config;

        set(imh, 'cdata', cat(3, cells, cells, z));

text(-10, 35.5, 'N', 'Rotation', 90, 'Color', 'k', 'FontSize', 17);
text(-10, 33, 'p', 'Rotation', 90, 'Color', 'k', 'FontSize', 17);
text(-10, 31, 'a', 'Rotation', 90, 'Color', 'k', 'FontSize', 17);

```

```

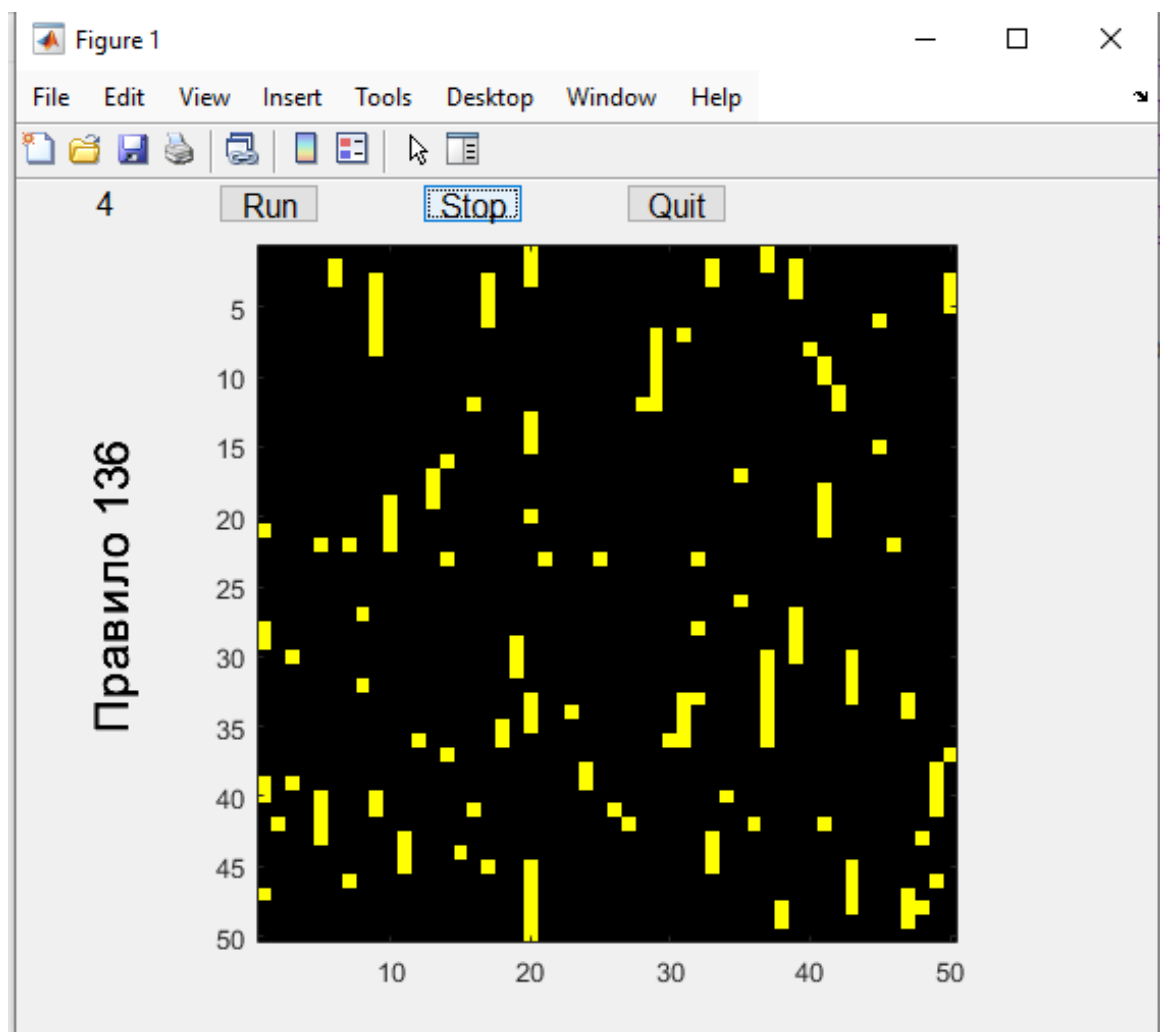
text(-10, 29, 'В', 'Rotation', 90, 'Color', 'k', 'FontSize', 17);
text(-10, 27, 'и', 'Rotation', 90, 'Color', 'k', 'FontSize', 17);
text(-10, 25, 'л', 'Rotation', 90, 'Color', 'k', 'FontSize', 17);
text(-10, 23, 'о', 'Rotation', 90, 'Color', 'k', 'FontSize', 17);
text(-10, 20, '136', 'Rotation', 90, 'Color', 'k', 'FontSize', 17);

    stepnumber = 1 + str2num(get(number, 'string'));
    set(number, 'string', num2str(stepnumber))
    pause(0.1);
end

if (freeze == 1)
    run = 0;
    freeze = 0;
end
drawnow
end

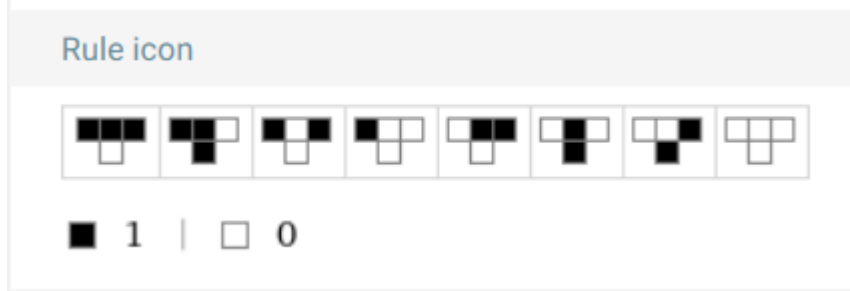
```

Виконання програми для правила 136:



Правило 70(01000110)

Можливі комбінації трьох сусідніх клітин на кроці t	Значення центральної клітини на кроці t + 1
111	0
110	1
101	0
100	0
011	0
010	1
001	1
000	0



Код програми для правила 70:

```
function new_state = apply_custom_rule(neighbors)
    % Custom Rule (Default: Rule 70)
    if neighbors(1) == 1 && neighbors(2) == 1 && neighbors(3) == 1
        new_state = 0;
    elseif neighbors(1) == 1 && neighbors(2) == 1 && neighbors(3) == 0
        new_state = 1;
    elseif neighbors(1) == 1 && neighbors(2) == 0 && neighbors(3) == 1
        new_state = 0;
    elseif neighbors(1) == 1 && neighbors(2) == 0 && neighbors(3) == 0
        new_state = 0;
    elseif neighbors(1) == 0 && neighbors(2) == 1 && neighbors(3) == 1
        new_state = 0;
    elseif neighbors(1) == 0 && neighbors(2) == 1 && neighbors(3) == 0
        new_state = 1;
    elseif neighbors(1) == 0 && neighbors(2) == 0 && neighbors(3) == 1
        new_state = 1;
    else
        new_state = 0;
    end
end

% GUI Initialization
clf
clear all

% Build the GUI
```

```

plotbutton = uicontrol('style','pushbutton',...
    'string','Run', ...
    'fontsize',12, ...
    'position',[100,400,50,20], ...
    'callback','run=1;');

erasebutton = uicontrol('style','pushbutton',...
    'string','Stop', ...
    'fontsize',12, ...
    'position',[200,400,50,20], ...
    'callback','freeze=1;');

quitbutton = uicontrol('style','pushbutton',...
    'string','Quit', ...
    'fontsize',12, ...
    'position',[300,400,50,20], ...
    'callback','stop=1;close;');
number = uicontrol('style','text', ...
    'string','1', ...
    'fontsize',12, ...
    'position',[20,400,50,20]);

% CA setup
n = 50; % Збільшення розмірів клітин
z = zeros(n,n);
cells = z;
sum = z;
cells(n/2, .25*n:.75*n) = 1;
cells(.25*n:.75*n, n/2) = 1;
cells = (rand(n,n)) < 0.5;

imh = image(cat(3, cells, cells, z)); % Сині клітини
set(imh, 'erasemode', 'none')
axis equal
axis tight

x = 2:n-1;
y = 2:n-1;

stop = 0;
run = 0;
freeze = 0;
stepnumber = 1;

while (stop == 0 && stepnumber <= 50)

    if (run == 1)
        updated_config = zeros(size(cells));
        for i = 1:n
            for j = 1:n
                neighbors = [cells(mod(i-2, n)+1, j), ...
                    cells(i, mod(j, n)+1), ...
                    cells(mod(i, n)+1, mod(j, n)+1)];
                updated_config(i, j) = apply_custom_rule(neighbors);
            end
        end
        cells = updated_config;

        set(imh, 'cdata', cat(3, cells, cells, z));

    text(-10, 35.5, 'П', 'Rotation', 90, 'Color', 'k', 'FontSize', 17);
    text(-10, 33, 'p', 'Rotation', 90, 'Color', 'k', 'FontSize', 17);

```

```

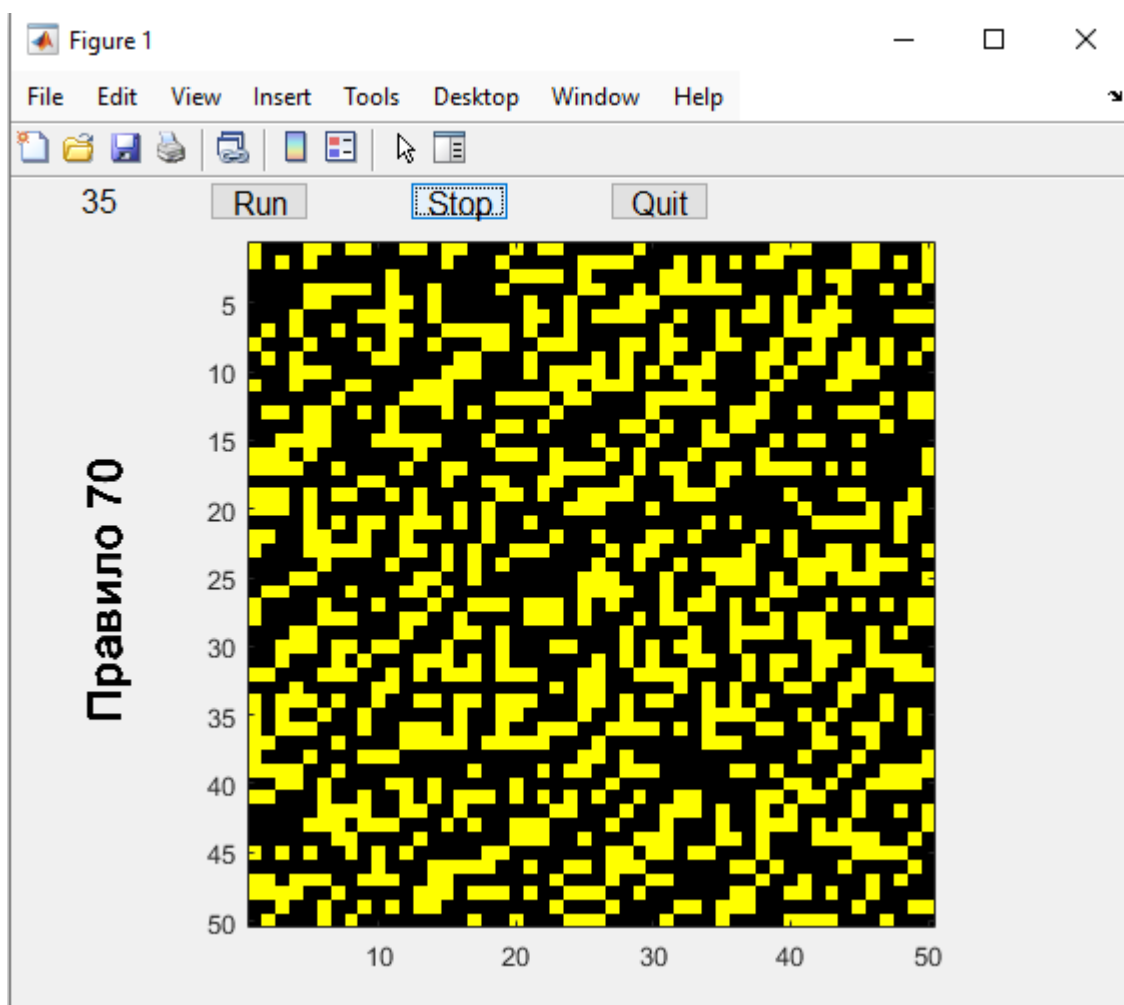
text(-10, 31, 'а', 'Rotation', 90, 'Color', 'k', 'FontSize', 17);
text(-10, 29, 'в', 'Rotation', 90, 'Color', 'k', 'FontSize', 17);
text(-10, 27, 'и', 'Rotation', 90, 'Color', 'k', 'FontSize', 17);
text(-10, 25, 'л', 'Rotation', 90, 'Color', 'k', 'FontSize', 17);
text(-10, 23, 'о', 'Rotation', 90, 'Color', 'k', 'FontSize', 17);
text(-10, 20, '70', 'Rotation', 90, 'Color', 'k', 'FontSize', 17);

    stepnumber = 1 + str2num(get(number, 'string'));
    set(number, 'string', num2str(stepnumber))
    pause(0.1);
end

if (freeze == 1)
    run = 0;
    freeze = 0;
end
drawnow
end

```

Виконання програми для правила 70:



Висновок

У рамках даної лабораторної роботи було опрацьовано теоретичні аспекти моделювання клітинних автоматів, розглянуто роботу програми Life в MATLAB, а також вивчено роботу гри Conway's Life.

Моделювання клітинних автоматів є важливою галуззю досліджень у комп'ютерних науках, оскільки воно дозволяє аналізувати поведінку складних систем через їхню взаємодію на рівні окремих компонентів. Клітинні автомати можуть бути використані для моделювання різних системних явищ, таких як рост рослин, розвиток тканин, фазові переходи та інші.

Гра "Життя" Конвея є класичним прикладом клітинного автомата, який демонструє емерджентні властивості системи. Вона базується на простих правилах, які визначають, як змінюється стан клітин з покоління в покоління. Дозволяє спостерігати за еволюцією простих структур згідно з цими правилами. Програма Conway's Life в MATLAB реалізує гру "Життя" за допомогою графічного інтерфейсу користувача. Вона використовує прості кнопки для керування відтворенням і зупинкою еволюції системи. Ця програма дозволяє візуалізувати еволюцію клітинного автомата і аналізувати його властивості за допомогою графічного інтерфейсу.

Загальна мета лабораторної роботи - дослідити різні правила клітинного автомата для "хаотичної" початкової конфігурації та спостерігати за їхньою еволюцією. Для досягнення цієї мети були використані різні правила, такі як правило 18, правило 73, правило 136 і правило Вольфрама з номером 70.

Отже, лабораторна робота дозволила краще розібратися з теоретичними аспектами клітинних автоматів, ознайомитися з реалізацією гри "Життя" в MATLAB і дослідити різні правила клітинного автомата на практиці.